CRIMEVISIONAI

Coding & Solution

Folder 5 | Implementation summary and reproducibility notes | Prepared 23 October 2026

Project: CrimeVisionAI — Multimodal Forensic Evidence Analysis and Investigation Support System

Document status: Project-specific draft based on the implementation information currently available. Verify placeholders and run-dependent results before formal submission.

1. Solution summary

Field	CrimeVisionAI details
Repository URL	[Add the actual GitHub repository URL before submission]
Languages	Python, JavaScript, HTML, CSS
Framework / UI	Flask 3.x app factory; Jinja2 templates; vanilla JavaScript frontend
Persistence	SQLite via raw sqlite3; local evidence file storage
AI / ML components	Whisper base, RT-DETR rtdetr-l.pt, CLIP ViT-B/32, BLIP, Groq openai/gpt-oss-20b
Core features	Case/evidence workflow; audio/image/video analysis; candidate events/timeline; case-level Q&A
Known gaps	Q&A grounding/completeness issue; no speaker diarization; PDF report export and 3D visualization not confirmed complete.

2. Implementation details

Audio: supported formats reported as WAV, MP3, M4A, FLAC, OGG, AAC and WMA. Whisper processes audio in 30-second chunks and emits timestamped transcription. Groq assists in structuring candidate timeline events; no speaker diarization is present in the known baseline. Image: RT-DETR object detection with confidence presets 0.25, 0.4, 0.5 and 0.6; CLIP compares the image with a set of scene labels; BLIP generates a caption; Groq generates cautious interpretation. Model scores and generated text are not proof of an event. Video: supports MP4, MOV, AVI, MKV and WEBM; samples at 1 FPS and caps processing at 180 seconds / 180 frames. Frame-level detections/captions/labels can contribute to timeline outputs. Case workflow: case and evidence APIs, media retrieval and case chat are reported. SQLite tables include cases, evidence and analysis_results.

3. Code quality checklist

Criterion	Status	Notes
Code modularity	Partially evidenced	App factory and modality-specific endpoints are reported; source-level review required.
Meaningful naming	To verify	Review actual functions and variables.
Comments / documentation	To verify	Check code comments and docstrings.
Critical error handling	Partially evidenced / test required	Run negative-path tests for upload, models, API and DB.
Application runs without critical errors	Known local baseline reported; re-run required	Perform clean-environment setup and smoke test.
Version control	To verify	Insert repository URL and confirm committed source.

4. Local run instructions (verify against actual repository)

1. Install a compatible Python version used by the repository. 2. Create and activate a virtual environment. 3. Install dependencies from the project's actual requirements.txt. 4. Configure required environment variables, including the Groq API key if the integration is enabled; do not paste secrets into documentation. 5. Ensure model weights are available or allow documented model downloads. 6. Initialize/confirm the SQLite database and required upload directories. 7. Run the documented entry point (reported as run.py) from the project root. 8. Open the local address reported by the application, currently 127.0.0.1:5000 in the known development configuration. 9. Run a smoke test using a small authorized/synthetic evidence file. 10. Do not expose the debug server to the public internet.

5. Pending / incomplete items

Confirm and fix investigator Q&A; grounding, especially cases where a relevant detected entity is omitted. Add repeatable tests for timeline timestamp alignment and evidence references. Verify whether PDF report generation exists in the latest repository. Confirm setup on a clean environment. Review security controls and disable debug mode before any shared deployment.